

Análisis de Algoritmos I

Algoritmos de optimización

María Janneth Rivera Reyna

Lic. En Ciencias de la Computación
Departamento de Matemáticas
Universidad de Sonora

Ciclo 2026-1

Algoritmos de optimización

- Son procedimientos matemáticos y computacionales diseñados para **encontrar la mejor solución posible** a un problema, entre un conjunto de opciones disponibles.
- Se define una **función objetivo**, la cual se desea **maximizar o minimizar**, según sea el caso, junto con un conjunto de **restricciones**:
 - **Maximizar**: Obtener la mayor ganancia, eficiencia, velocidad o rendimiento (ej. armar el equipo de futbol con más puntos).
 - **Minimizar**: Reducir al máximo el costo, el tiempo de entrega, el desperdicio de material o el riesgo (ej. encontrar la ruta más corta en el GPS).

Algoritmos de optimización: Analogía – Equipo de Futbol

- **Objetivo:** Armar el equipo más barato para no quebrar (minimizar costo).
- **Restricciones:** Contratar a 11 jugadores y no pasarte del presupuesto.
- **Optimización:** Elegir a los jugadores que cumplan con las reglas y me cueste lo más barato.

Equipo	¿Es Factible? (Cumple reglas)	Costo (Función Objetivo)	¿Es Óptimo?
11 estrellas mundiales	❌ NO (Excede presupuesto)	❌ \$1,000M	❌ No
11 jugadores promedio	✅ SÍ	❌ \$50M	❌ No (Hay más baratos)
➡ 11 jugadores novatos	✅ SÍ	✅ \$10M	✅ SÍ (Es el mínimo)
5 jugadores (Faltan 6)	❌ NO (No cumple las reglas de la liga)	✅ \$5M	❌ No

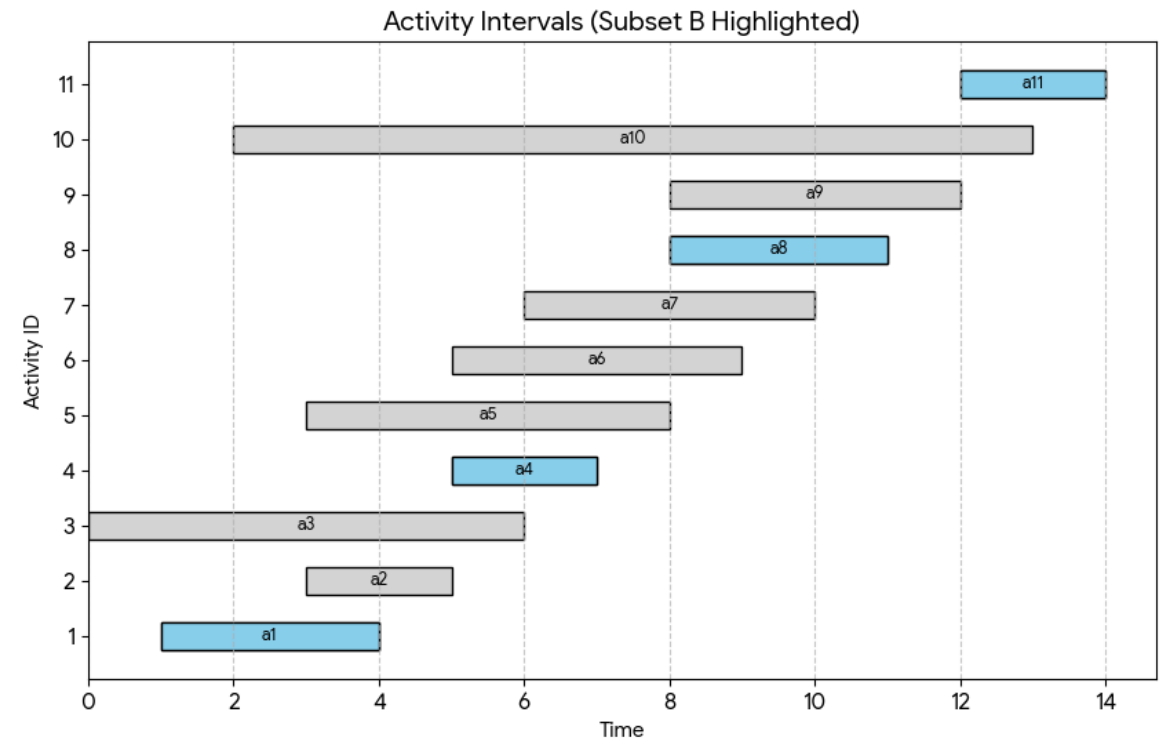
Algoritmos Voraces (Greedy)

Pasos para desarrollar un algoritmo voraz:

1. Definir la **subestructura óptima** del problema.
2. Desarrollar una **solución recursiva** (solución lógica, sin programar aún).
3. Mostrar que si se toma una decisión voraz, entonces **sólo resta un subproblema que resolver**.
4. Probar que siempre **es seguro tomar una decisión voraz**.
5. Implementar un **algoritmo recursivo** que utilice la estrategia voraz.
6. Convertir el algoritmo recursivo a uno **iterativo**.

Selección de actividades (Algoritmo voraz)

- El problema consiste en seleccionar el **número máximo de actividades** que puede realizar una sola persona (o máquina), suponiendo que solo puede trabajar en una actividad a la vez (no overlap), dentro de un tiempo determinado.



Selección de actividades: Definición

- Suponga que se tiene un conjunto $S = \{a_1, a_2, a_3, \dots, a_n\}$ de **n actividades**
- Cada actividad a_i tiene un **tiempo de inicio s_i** y un **tiempo de fin f_i** , donde $0 \leq s_i \leq f_i \leq \infty$.
- Una actividad seleccionada a_i tomará lugar en el intervalo de tiempo $[s_i, f_i)$.
- Las actividades a_i y a_j son **compatibles** si los intervalos $[s_i, f_i)$ y $[s_j, f_j)$ no se superponen (overlap).
- Esto quiere decir que, a_i y a_j son **compatibles** si $s_i \geq f_j$ ó $s_j \geq f_i$

Selección de actividades: Definición

- Se asume que las actividades están ordenadas en orden creciente por tiempo de finalización:

$$f_1 \leq f_1 \leq f_1 \leq \dots \leq f_{n-1} \leq f_n$$

Ejemplo: Considere el siguiente conjunto de actividades S

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

- El subconjunto $\{a_3, a_9, a_{11}\}$ contiene actividades mutuamente compatibles.
- No es el subconjunto máximo, ya que $\{a_1, a_4, a_8, a_{11}\}$ es más grande. De hecho, este sería uno de los más grandes, otro sería $\{a_2, a_4, a_9, a_{11}\}$.

Selección de actividades: Diseño de la solución voraz

1. **Determinar la estructura óptima:** Si elegimos una actividad a_i , la solución óptima total consistirá en esa actividad a_i más la solución óptima para el conjunto de actividades restantes que no se empalman con ella.
2. **Desarrollar una solución recursiva:** Definimos el problema como una función que busca el conjunto máximo entre un tiempo t :
 - Para el tiempo restante después de la última actividad, busca todas las actividades que empiezan después y elige una. Repite el proceso. (Aquí todavía no decimos cuál elegir, solo que el proceso se repite).
3. **Mostrar que solo queda un subproblema tras la elección voraz:** Dado que las actividades están ordenadas de forma creciente por tiempo de finalización, la elección voraz es la actividad que termina primero: a_1 . Entonces, solo nos queda un subproblema por resolver: buscar actividades que comiencen después de que a_1 termine.

Selección de actividades: Diseño de la solución voraz

4. Probar que la elección voraz es siempre segura:

- Supongamos que existe una solución perfecta que no empieza con la actividad que termina más pronto (a_1), sino con otra actividad (a_k).
- Como a_1 termina antes o al mismo tiempo que a_k , podemos quitar a a_k y poner a a_1 en su lugar sin chocar con ninguna otra actividad de la lista.
- El resultado sigue siendo igual de bueno (mismo número de actividades). Por lo tanto, elegir la que termina primero es una apuesta segura*.

*Cormen demuestra formalmente con el teorema 16.1 que existe una solución óptima para el problema que comienza con la actividad que tiene el tiempo de finalización más temprano (la elección voraz).

Selección de actividades: Pseudocódigo algoritmo recursivo voraz

Algoritmo #1 RECURSIVE-ACTIVITY-SELECTOR

Input:

s: arreglo de tiempos de inicio
f: arreglo de tiempos de finalización
k: el índice del subproblema a resolver S_k
n: el tamaño del problema original

Output: un conjunto de tamaño máximo de actividades mutuamente compatibles en S_k .

RECURSIVE-ACTIVITY-SELECTOR(s, f, k, n)

m = k + 1

Empezamos revisando la actividad siguiente (m) a la que ya seleccionamos (k). Si es la primera vez, se usa una actividad ficticia a_0

Descartar actividades hasta encontrar una compatible (que empiece después de que termine la actual).

while m ≤ n and s[m] < f[k]
 m = m + 1

Mientras haya actividades en la lista **Y** la actividad actual (m) empiece antes de que termine la anterior (k), simplemente la “saltamos” (m=m+1) porque no es compatible y seguimos buscando.

if m < n

return { a_m } ∪ RECURSIVE-ACTIVITY-SELECTOR(s, f, m, n)

else return []

Selección de Actividades: Análisis de la implementación recursiva

Asumiendo que **las actividades han sido ordenadas** previamente por el tiempo de finalización, el tiempo de ejecución de la llamada recursiva `RECURSIVE-
ACTIVITY-SELECTOR(s, f, 0, n)` presenta una **complejidad temporal**:

$$O(n)$$


En todas las llamadas recursivas, cada actividad se examina exactamente una vez en el bucle `while`.

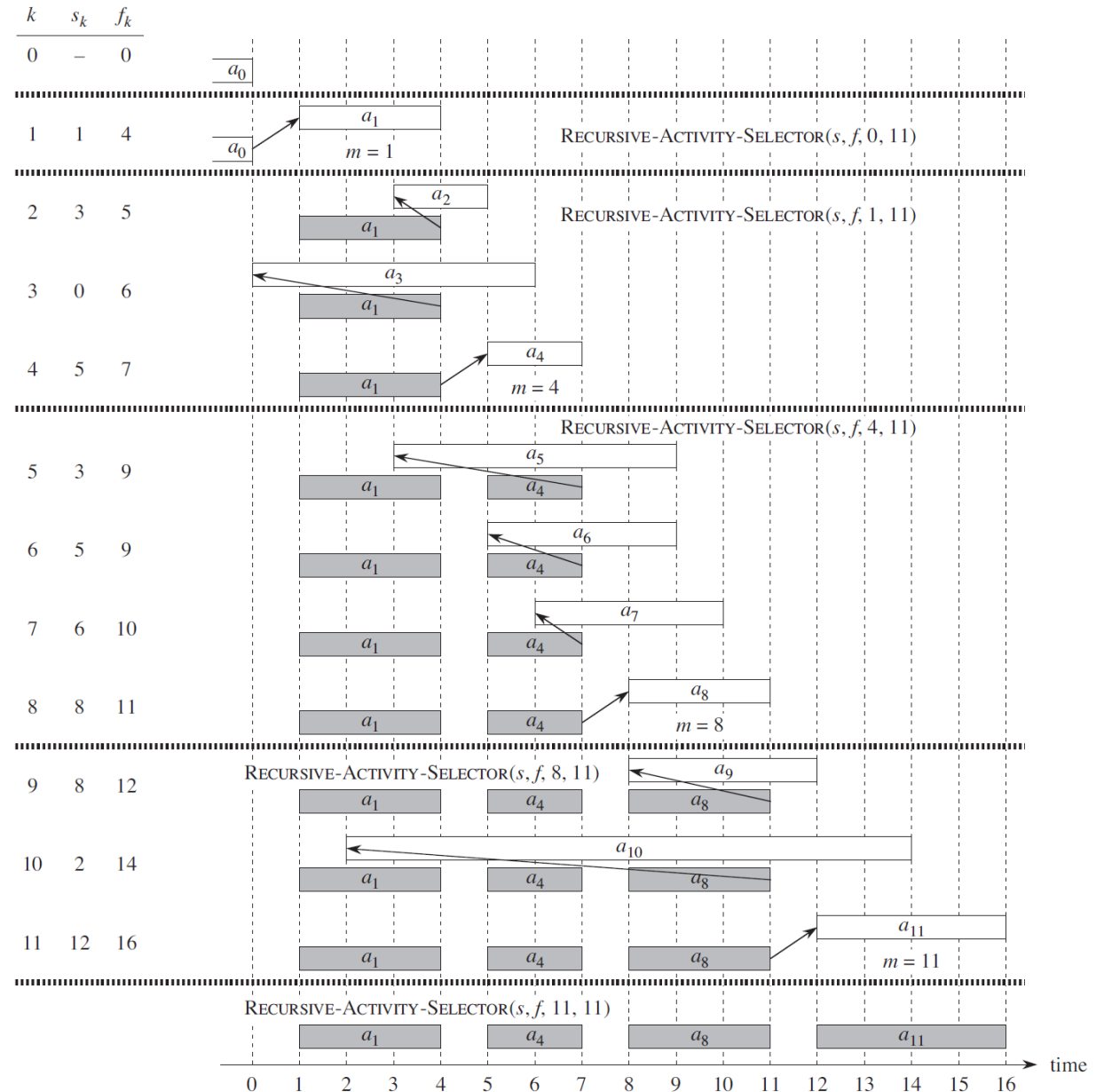
En cuanto a la **complejidad espacial**, la versión recursiva tomará $O(n)$, esto debido a la pila de llamadas que se genera si seleccionamos muchas actividades.

Selección de Actividades: Algoritmo recursivo voraz

i	1	2	3	4	5	6	7	8	9	10	11
s_i	1	3	0	5	3	5	6	8	8	2	12
f_i	4	5	6	7	9	9	10	11	12	14	16

El conjunto resultante de actividades seleccionadas es:

$\{a_1, a_4, a_8, a_{11}\}$



Selección de actividades: Algoritmo iterativo voraz

Algoritmo #1 GREEDY-ACTIVITY-SELECTOR

Input:

s: arreglo de tiempos de inicio

f: arreglo de tiempos de finalización

Output: un conjunto A de tamaño máximo de actividades compatibles.

GREEDY-ACTIVITY-SELECTOR(s, f)

n = s.length

A = { a_1 }

k = 1

Buscar la primera actividad que inicia después de que termine la actual

for m = 2 **to** n

if s[m] >= f[k]

 A = A $\cup$ { a_m }

 k = m

return A

Selección de Actividades: Análisis de la implementación iterativa

Al igual que la versión recursiva, y **asumiendo que las actividades fueron ordenadas** de manera creciente por el tiempo de finalización, entonces el tiempo de ejecución de la implementación iterativa `GREEDY-ACTIVITY-SELECTOR(s, f)` presenta una **complejidad temporal**:

$$\underline{O(n)}$$

En cuanto a la **complejidad espacial**, la versión iterativa tomará $O(1)$, ya que solo usamos un par de variables para rastrear la última actividad y el índice actual.

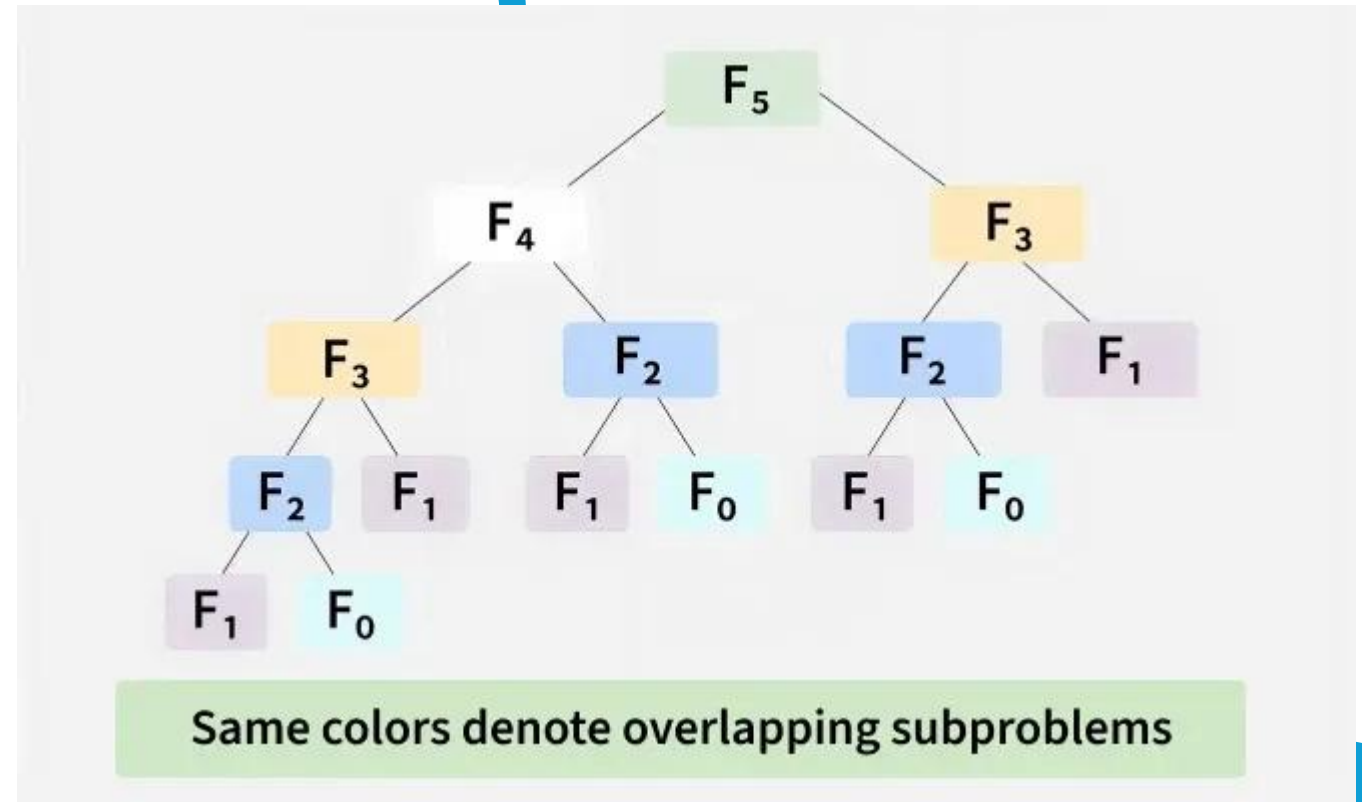
Selección de Actividades: Resumen del análisis de complejidad temporal

Caso	Complejidad $O()$
Mejor caso (actividades ordenadas)	$O(n)$
Caso promedio	$O(n \log n)$
Peor caso (hay que ordenar primero)	$O(n \log n)$

Programación Dinámica (DP)

Secuencia Fibonacci

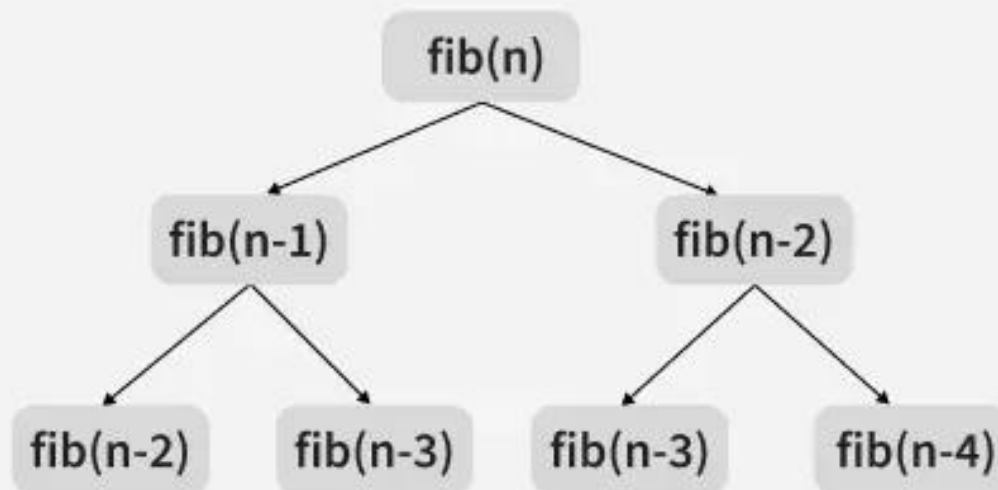
- La sucesión de Fibonacci es: 0, 1, 1, 2, 3, 5, 8, 13, 21...
- Dado un entero positivo n , encontrar el n -ésimo número de Fibonacci. La sucesión de Fibonacci es una secuencia donde cada término es la suma de los dos términos anteriores. Los dos primeros términos de la sucesión de Fibonacci son 0,1.



Secuencia Fibonacci: recursivo original

Recursion: Exponential

Fibonacci sequence using a recursion tree, showing the exponential time complexity of the naive approach.



```
int fib(int n) {  
    if (n <= 1)  
        return n;  
    return fib(n-1) + fib(n-2);  
}
```

Algoritmo #2.1 – Fibonacci recursivo original

Input: índice n del elemento que deseamos encontrar dentro de la secuencia de Fibonacci.

Output: valor numérico que corresponde a esa posición en la secuencia.

```
def fib_recursive(n) :  
    if n <= 1:  
        return n  
    return fib_recursive(n - 1) + fib_recursive(n - 2)
```

Secuencia Fibonacci: Análisis versión recursiva original

- Supongamos que queremos resolver $fib(5)$, producimos un árbol de llamadas que invoca la función sobre el mismo valor muchas veces diferentes:

1. `fib(5)`
2. `fib(4) + fib(3)`
3. `(fib(3) + fib(2)) + (fib(2) + fib(1))`
4. `((fib(2) + fib(1)) + (fib(1) + fib(0))) + ((fib(1) + fib(0)) + fib(1))`
5. `((((fib(1) + fib(0)) + fib(1)) + (fib(1) + fib(0))) + ((fib(1) + fib(0)) + fib(1)))`

- En particular, $fib(2)$ se calculó 3 veces desde cero. En ejemplos más grandes, se recalculan muchos más valores de fib , o subproblemas, lo que da como resultado un algoritmo de **complejidad temporal exponencial**:

$$\underline{O(2^n)}.$$

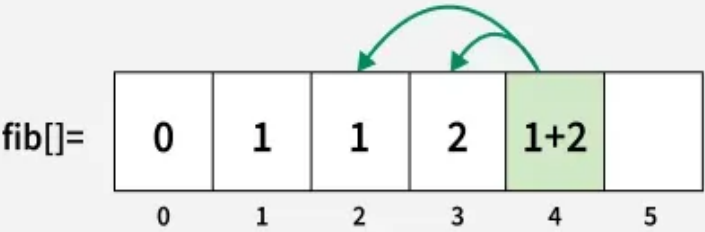
- Mientras que la **complejidad espacial** será de complejidad lineal dado el espacio que requiere la recursión $O(n)$.

Secuencia Fibonacci: bottom-up tabulation

- Ahora bien, supongamos que tenemos un arreglo *fib*, que almacena cada valor que ya se ha calculado, y modificamos nuestra función para usarlo y actualizarlo.
- Con este enfoque, calculamos primero los valores más pequeños de *fib*, y luego construimos los más grandes a partir de ellos.

Dynamic Programming: Linear

Fibonacci sequence is optimized using the tabulation (bottom-up) approach, which reduces the time complexity to linear.



fib[] =

0	1	1	2	1+2	
0	1	2	3	4	5

```
fib[0] = 0;
fib[1] = 1;
for (i = 2; i <= n; i++) {
    fib[i] = fib[i-1] + fib[i-2];
}
return fib[n];
```

Dynamic Programming or DP

Algoritmo #2.2 – Fibonacci iterativo bottom-up tabulation

Input: índice n del elemento que deseamos encontrar dentro de la secuencia de Fibonacci.

Output: valor numérico que corresponde a esa posición en la secuencia.

```
def fib_tabulation(n):  
    if n <= 1:  
        return n  
  
    # Creamos la tabla (arreglo)  
    fib = [0] * (n + 1)  
    fib[0] = 0  
    fib[1] = 1  
  
    for i in range(2, n + 1):  
        fib[i] = fib[i - 1] + fib[i - 2]  
  
    return fib[n]
```

Secuencia Fibonacci: Análisis bottom-up tabulation

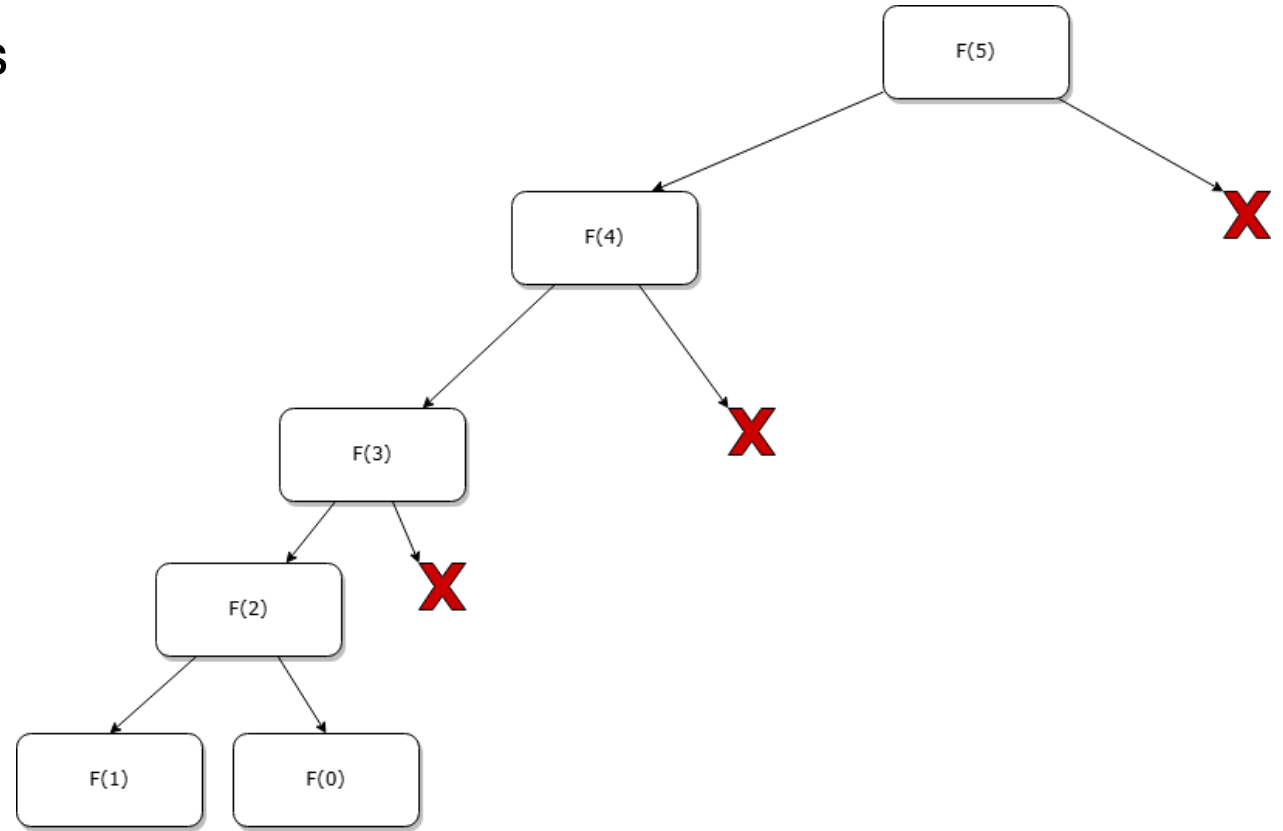
- La función resultante tendrá una complejidad temporal de:

$$\underline{O(n)}$$

- La complejidad espacial estará dada por $O(n)$, dado el espacio que ocupa la pila de llamadas de la recursión y la tabla de resultados almacenados.

Secuencia Fibonacci: top-down memoization

- En este enfoque primero dividimos el problema en subproblemas y luego calculamos y almacenamos los valores.
- Supongamos que tenemos un objeto (un diccionario, lista, hash map), que guarda cada resultado ya calculado, y modificamos nuestra función para usarlo y actualizarlo.



Algoritmo #2.3 – Fibonacci recursivo DP Top-down memoization

Input: índice n del elemento que deseamos encontrar dentro de la secuencia de Fibonacci.

Output: valor numérico que corresponde a esa posición en la secuencia.

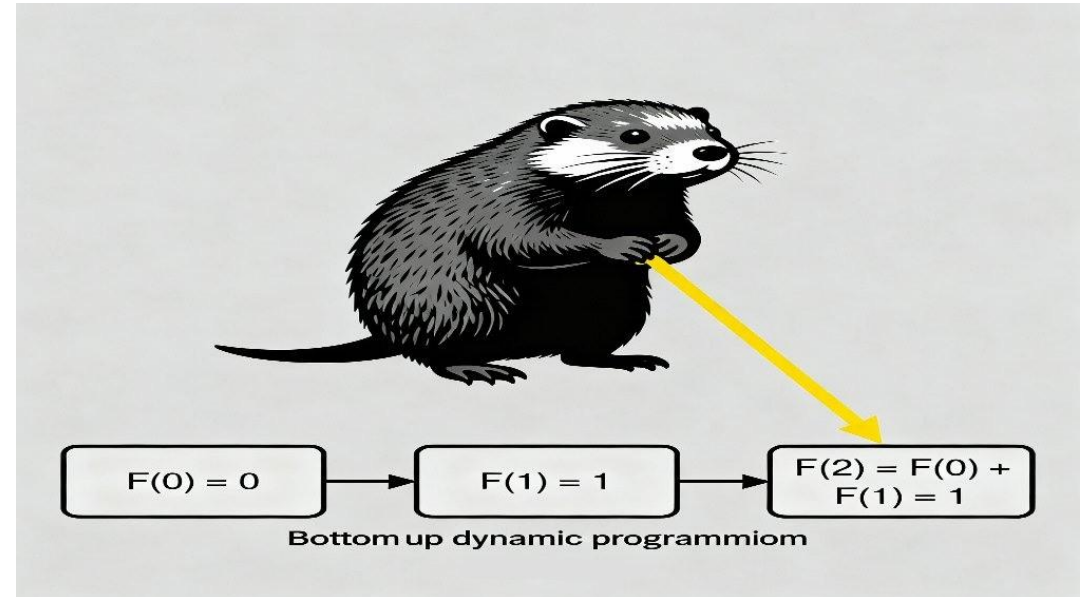
```
def fib_memoization(n, memo=None):  
    if memo is None:  
        memo = [-1] * (n + 1)  
  
    # ¿Ya lo calculamos?  
    if memo[n] != -1:  
        return memo[n]  
  
    # Caso base  
    if n <= 1:  
        memo[n] = n  
    else:  
        # Guardamos el resultado antes de retornarlo  
        memo[n] = fib_memoization(n - 1, memo) + fib_memoization(n - 2, memo)  
    return memo[n]
```

Secuencia Fibonacci: Análisis top-down memoization

- Esta implementación muestra una complejidad temporal de $O(n)$
- Mientras que la complejidad espacial será de $O(n)$, por la pila de recursión y el objeto que almacena los valores ya calculados.

Secuencia Fibonacci: Análisis bottom-up tabulation [Optimizado]

- Sabemos que cualquier número de Fibonacci depende únicamente de los **dos valores anteriores**.
- Por lo tanto, no necesitamos almacenar la tabla completa de tamaño $n+1$, sino que podemos almacenar **solo los dos valores anteriores en variables**.



Algoritmo #2.4 – Fibonacci iterativo DP bottom-up tabulation OPT

Input: índice n del elemento que deseamos encontrar dentro de la secuencia de Fibonacci.

Output: valor numérico que corresponde a esa posición en la secuencia.

```
def fib_optimized(n) :  
    if n <= 1:  
        return n  
  
    prev2 = 0    # fib[i-2]  
    prev1 = 1    # fib[i-1]  
  
    for _ in range(2, n + 1):  
        current = prev1 + prev2  
        # Desplazamos los valores para la siguiente iteración  
        prev2 = prev1  
        prev1 = current  
  
    return prev1
```

Secuencia Fibonacci: Análisis bottom-up tabulation [Optimizado]

- De esta forma, la **complejidad temporal** será de igual forma lineal

$$\underline{O(n)}$$

- La diferencia radica en la **complejidad espacial**, la cual ahora solo requerirá de un espacio adicional constante $O(1)$.

Secuencia Fibonacci: Resumen

Implementación	Complejidad temporal	Complejidad espacial
Recursiva original	$O(2^n)$	$O(n)$
Bottom-up tabulation	$O(n)$	$O(n)$
Top-down memoization	$O(n)$	$O(n)$
Top-down memoization optimized	$O(n)$	$O(1)$

Bibliografía

- Lee, R.C.T., Tseng, S. S., Chang, R. C. y Tsai, Y. T. (2007). Introducción al diseño y análisis de algoritmos. Un enfoque estratégico. México, D.F. Mc Graw Hill.
- Cormen T. H., Leiserson C. E., R. L. Rivest y C. S. (2022). *Introduction to Algorithms* .
- Cormen, T. H. (2013). *Algorithms Unlocked*.
- https://es.wikipedia.org/wiki/Algoritmo_voraz
- <https://www.youtube.com/watch?v=ALtJncFD8N8>
- <https://www.geeksforgeeks.org/python/dynamic-programming-in-python/>
- <https://www.geeksforgeeks.org/dsa/introduction-to-dynamic-programming-data-structures-and-algorithm-tutorials/>
- https://en.wikipedia.org/wiki/Dynamic_programming